Reviewer Pack — Sturmian Factor-Complexity Lower-Bounds DNF-MCSP Term Count

Entry 848aa89a6626 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 13:49:34 UTC

Sturmian Factor-Complexity Lower-Bounds DNF-MCSP Term Count

Entry ID: `848aa89a6626` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 13:49:34 UTC

1. Conjecture

Field A (mathematical branch): Symbolic dynamics on the Boolean hypercube: subword/factor complexity p_w(k) of the Gray-code linearization of a truth table, in the Morse-Hedlund / Sturmian tradition (rarely deployed in meta-complexity) **Field B** (complexity object): DNF-MCSP: minimum number of product terms DNF_min(f) needed to represent a Boolean function f:{0,1}^n→{0,1} given its 2^n-bit truth table (Hirahara–Oliveira–Santhanam restricted-MCSP variant)

Statement:

> Let w(f) be the binary word of length 2^n obtained by listing f's truth table in standard reflected Gray-code order, and let P(f)=max_{1≤k≤n} p_{w(f)}(k)/k where p_w(k) is the number of distinct length-k factors of w. Then DNF_min(f) ≥ ⌈P(f)/2⌉ for every non-constant f on n≤6 variables, with equality achieved by every parity function χ_S restricted to its essential variables. Equivalently: any f whose Gray-code word realizes factor-complexity ratio P(f)>2T cannot be expressed as a DNF with ≤T terms.

Rationale (proposer's reasoning):

> Gray-code traversal makes adjacent truth-table positions differ in exactly one input bit, so a single AND-term contributes a contiguous arithmetic-progression-like block to w(f); bounding the number of distinct factors therefore bounds the combinatorial diversity of term-boundaries, which in turn bounds DNF_min. Symbolic dynamics has been heavily used in automata theory and formal languages but, to our knowledge, never linked to MCSP-style term-minimization; Sturmian/Morse-Hedlund growth gives a clean, computable scalar invariant. Because the bound is a simple counting inequality on a 2^n-bit word it sidesteps relativization (the mapping is purely syntactic on the truth table).

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f4133adbbe2b6408`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥5000 non-constant Boolean functions on n∈{3,4,5,6} (full enumeration for n≤4, ≥5000 random samples for n∈{5,6}), compute P(f) and DNF_min(f). Conjecture is supported iff DNF_min(f) ≥ ⌈P(f)/2⌉ holds for ≥99% of f AND equality holds for every parity χ_S on essential variables.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Gray code truth table subword complexity DNF minimization` - `factor complexity Boolean function DNF lower bound Morse-Hedlund` - `Sturmian subword complexity meta-complexity MCSP Boolean circuit`

Top relevant hits considered: - [http://arxiv.org/abs/1303.5057v1] GRay: a Massively Parallel GPU-Based Code for Ray Tracing in Relativistic Spacetimes - [http://arxiv.org/abs/1203.2323v2] The greedy flip tree of a subword complex - [http://arxiv.org/abs/2412.05161v1] DNF: Unconditional 4D Generation with Dictionary-based Neural Fields - [http://arxiv.org/abs/1501.01331v1] DNF complexity of complete boolean functions - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/2404.02414v2] A simple lower bound for the complexity of estimating partition functions on a quantum computer - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1212.3064v1] Subword complexity and Sturmian colorings of regular trees

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=180.1s

5.1 Generated Python source

```python
import random
import math
import json
from itertools import product

def truth_table(f, n):
    inputs = list(product([0, 1], repeat=n))
    outputs = [f(tuple(inputs[i])) for i in range(len(inputs))]
    return outputs

def gray_code(n):
    if n == 0:
        return ['']
    prev_gray = gray_code(n-1)
    return ['0' + code for code in prev_gray] + ['1' + code for code in reversed(prev_gray)]
```

```python
def factor_complexity(word):
    n = len(word)
    p_w = [0] * (n+1)
    for k in range(1, n+1):
        factors = set()
        for i in range(n-k+1):
            factors.add(word[i:i+k])
        p_w[k] = len(factors)
    return max(p_w) / n

def dnf_min(f, n):
    tt = truth_table(f, n)
    prime_implicants = []
    for i in range(2**n):
        if tt[i]:
            implicant = [i]
            for j in range(i+1, 2**n):
                if (tt[j] and all((j & (1 << k)) == (i & (1 << k)) for k in range(n))):
                    implicant.append(j)
            prime_implicants.append(implicant)
    terms = []
    while prime_implicants:
        selected = None
        for i, imp in enumerate(prime_implicants):
            if not any(all((j & (1 << k)) == (i & (1 << k)) for k in range(n)) for j in imp for ot
                selected = i
                break
        if selected is None:
            break
        terms.append(prime_implicants[selected])
        prime_implicants = [imp for i, imp in enumerate(prime_implicants) if not any(all((j & (1 <
    return len(terms)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [3, 4, 5, 6]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        functions = [(lambda f: lambda x: f(x)) for _ in range(2**n)]
        random.shuffle(functions)
        for f in functions[:5000]:
            w_f = ''.join(str(bit) for bit in truth_table(f, n))
            p_w_f = factor_complexity(w_f)
            DNF_min_f = dnf_min(f, n)
            metric_value = DNF_min_f >= math.ceil(p_w_f / 2)
            total_metric_value += int(metric_value)
            instances_tested += 1
            if not metric_value:
                conjecture_holds = False
                counterexample = f"n={n}, function: {f}"
```

```
    return {
        "metric_name": "DNF_min >= ceil(P(f)/2)",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    total_metric_value = sum(res["metric_value"] * res["instances_tested"] for res in results) / s
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value} std=0.0 support_fraction={support_frac
    elif any(not res["conjecture_holds"] for res in results) and any(res["counterexample"] for res
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 180s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out after 180s without producing any RESULT line, so the pre-registered support condition cannot be evaluated and the critic's challenge stands. | next: Optimize the DNF_min computation (e.g., use Quine-McCluskey with caching or an ILP solver) and reduce sample sizes or parallelize so the test completes within the timeout.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	19930
2	preregistration	claude_max	opus	0	0	6213
3	novelty	claude_max	opus	0	0	3255
4	novelty	claude_max	opus	0	0	7995
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10831
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11677
7	judge	claude_max	opus	0	0	5026

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 64927 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/848aa89a6626.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/848aa89a6626.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/848aa89a6626.tar.gz` (if generated)